Atividade de IC-PICME: PVI Motivação

Sophia Damm Zogaib Mardones
Universidade Estadual de Campinas (UNICAMP)

6 de novembro de 2026

Resumo

[FAZER]

1 Introdução

> *"Não importa o quão bela seja a sua teoria, não importa o quão inteligente você seja. Se ela não concorda com a experiência, ela está errada."* — Richard Feynman

A análise numérica clássica de Problemas de Valor Inicial (PVIs) repousa sobre alicerces teóricos de profunda elegância. Munidos de teoremas clássicos de convergência [2, 1], aprendemos que, desde que um método seja consistente e estável, a matemática garante que a solução aproximada encontrará a solução exata. No papel, essas demonstrações são belas e inquestionáveis. Contudo, como adverte Feynman, a beleza de uma teoria encontra o seu verdadeiro limite na experiência: na matemática aplicada, a experiência é a execução computacional. Existe um abismo entre a utopia teórica de um passo infinitesimal ($k \to 0$) e a realidade incontornável de um algoritmo que precisa avançar em passos finitos ($k > 0$).

Neste abismo, o computador atua como o meio final. Ele não é um artífice de milagres capaz de consertar as nossas escolhas, mas sim um executor implacável e cegamente fiel à formulação que lhe é imposta. É nesse contexto que a célebre provocação de Pablo Picasso de que *"computadores são inúteis, eles só conseguem te dar respostas"* ganha um significado literal. A máquina devolverá, com obediência irrestrita, a resposta exata produzida pelo método numérico implementado. Todavia, se o algoritmo adotado ignorar a topologia real do problema, essa resposta não passará de uma perigosa ilusão matemática.

Este trabalho propõe, assim, uma reflexão crítica sobre essa delicada interação entre teoria, método e máquina através da investigação de um PVI proposto. Ao longo de quatro etapas, investiga-se: no **Item (a)**, [preencher]; no **Item (b)**, [preencher]; no **Item (c)**, [preencher]; e, no **Item (d)**, [preencher].

1.1 O Problema de Valor Inicial Proposto

O objeto central deste estudo é o PVI não linear do tipo equação de Riccati [1, 2]:

$$y'(x) = y(x)^2 - g(x), \qquad y(0) = 2, \qquad g(x) = \frac{x^4 - 6x^3 + 12x^2 - 14x + 9}{(1+x)^2}, \tag{1}$$

cuja solução analítica exata é dada por

$$y(x) = \frac{(1-x)(2-x)}{1+x} = x - 4 + \frac{6}{1+x}. \tag{2}$$

Verificação e estrutura diferencial. Derivando (2), obtêm-se as derivadas de primeira, segunda e m-ésima ordens da solução exata:

$$y'(x) = 1 - \frac{6}{(1+x)^2} = \frac{x^2+2x-5}{(1+x)^2}, \qquad y''(x) = \frac{12}{(1+x)^3}, \qquad y^{(m)}(x) = \frac{(-1)^m\, 6\, m!}{(1+x)^{m+1}} \quad (m \geq 2). \tag{3}$$

Como $(x^2-3x+2)^2 = x^4-6x^3+13x^2-12x+4$, verifica-se imediatamente que

$$y(x)^2 - y'(x) = \frac{(x^4-6x^3+13x^2-12x+4)-(x^2+2x-5)}{(1+x)^2} = \frac{x^4-6x^3+12x^2-14x+9}{(1+x)^2} = g(x).$$

1.2 Notação e Ambiente Computacional

Ao longo das seções seguintes, a variável independente é denotada por t (com $t \equiv x$ na Eq. (1)), o passo uniforme por k, os nós da malha por $t_n = t_0 + nk$ e a aproximação numérica de $y(t_n)$ por U_n, seguindo a notação de LeVeque [2].

As simulações foram desenvolvidas em Python 3.12, utilizando as bibliotecas científicas NumPy (alocação e operações vetorizadas), SciPy (`scipy.optimize.newton` para resolução não linear nos esquemas implícitos) e Matplotlib 3.11 (geração gráfica), executadas em ambiente Linux (kernel 6.14, arquitetura `x86_64`) equipado com processador Intel Core i5-1135G7 (2,40 GHz). A implementação foi estruturada de forma modular no pacote `src/pvi/` — contendo os módulos `problemas.py` (definição da classe `PVI` e função `motivacao`), `metodos.py` (integradores numéricos), `uteis.py` (cálculo de erros, ordens observadas e exportação automatizada de tabelas `.csv` e `.tex`) e `plotagem.py` —, enquanto os roteiros reprodutíveis de cada item residem no diretório `experimentos/`.

Todo código fonte produzido e utilizado está disponível em: https://github.com/sophiadamm/ic-picme/tree/main/02-pvi-motivacao

Organização do repositório e fluxo de trabalho. A arquitetura do projeto e o encadeamento dos dados entre os módulos foram estruturados para assegurar modularidade, desacoplamento e reprodutibilidade estrita. O fluxo operacional e a hierarquia dos diretórios são sintetizados no Esquema 1.

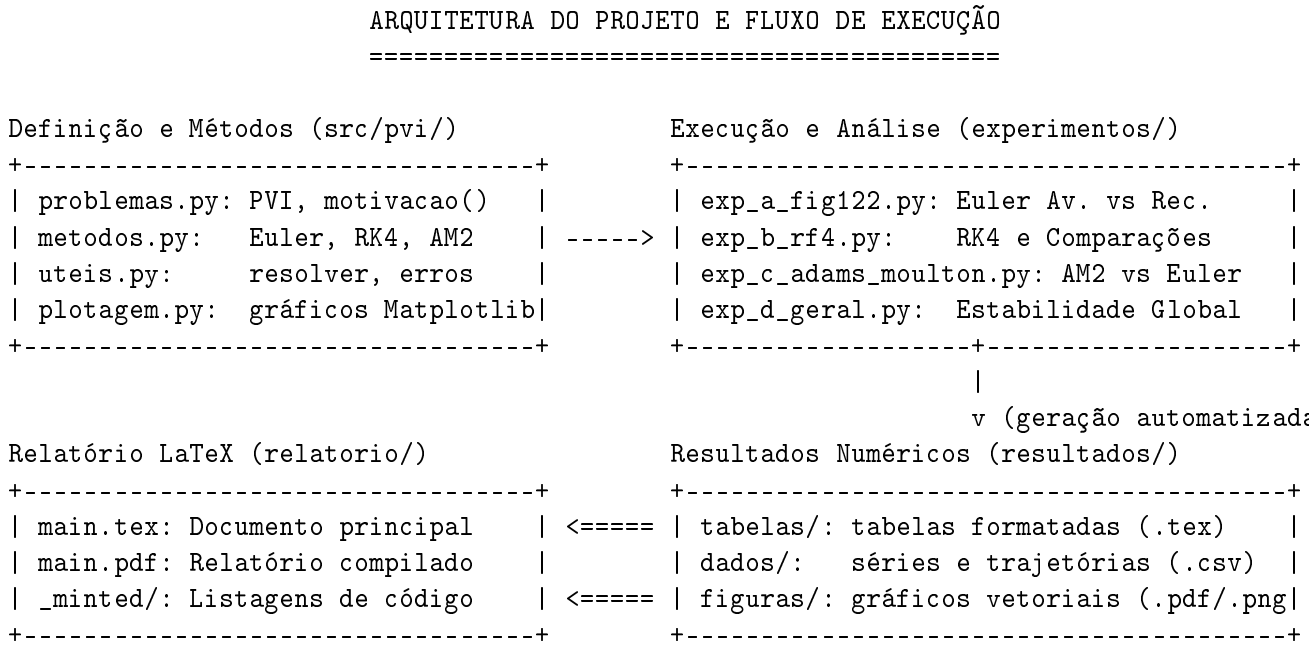


Figura 1: *Fluxo de trabalho computacional e organização das pastas do projeto: os módulos em* `src/pvi/` *fornecem o núcleo dos métodos; os roteiros em* `experimentos/` *executam as simulações e geram automaticamente saídas em* `resultados/`*; os arquivos* `.tex` *e figuras vetoriais são incorporados diretamente na compilação do relatório em* `relatorio/main.tex`*.*

2 Item (a): Euler avançado e Euler recuado

2.1 Objetivo

Implementar os métodos de Euler avançado (explícito) e Euler recuado (implícito) e reproduzir a Figura 12.2 de Süli e Mayers [1] para o PVI de Motivação no intervalo [0, 1,6], com passos $k = 0{,}2$, 0,1 e 0,05. Além da reprodução, o objetivo é *explicar* o comportamento observado.

2.2 Fundamentação Teórica

Para iniciar as tentativas e análise do PVI (Eq. (1)), recorremos inicialmente ao método mais clássico: O Método de Euler. Para obtê-lo, podemos escrever o PVI na forma $y' = f(t, y)$, com $f(t, y) = y^2 - g(t)$. Pela interpretação com o cálculo integral, obtemos a identidade exata

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f\big(s, y(s)\big)\, ds. \tag{4}$$

Os dois métodos de Euler correspondem a aproximar essa integral pela regra do retângulo, usando o valor do integrando em uma das extremidades:

$$\text{Euler avançado (explícito):} \quad U_{n+1} = U_n + k\, f(t_n, U_n), \tag{5}$$

$$\text{Euler recuado (implícito):} \quad U_{n+1} = U_n + k\, f(t_{n+1}, U_{n+1}). \tag{6}$$

A diferença estrutural é que em (6) a incógnita U_{n+1} aparece dos dois lados: a cada passo é preciso resolver uma equação (em geral não linear) para avançar.

Convergência: Consistência e Zero-Estabilidade Substituindo a solução exata nos esquemas e expandindo em série de Taylor, é possível concluir que os erros de truncamento locais são [2, 1]

$$\tau_n^{\mathrm{FE}} = \frac{k}{2}\, y''(\xi_n), \qquad \tau_n^{\mathrm{BE}} = -\frac{k}{2}\, y''(\eta_n), \qquad \xi_n, \eta_n \in (t_n, t_{n+1}). \tag{7}$$

Ambos os métodos são, portanto, consistentes de ordem 1. O ponto importante, que será decisivo neste problema, é que os dois erros locais têm a *mesma magnitude* mas *sinais opostos*: se $y'' > 0$, o Euler avançado tende a produzir valores abaixo da solução exata e o Euler recuado, valores acima.

A **zero-estabilidade** decorre da continuidade de Lipschitz de f em y (LeVeque [2], Cap. 5). Aqui $f(x, y) = y^2 - g(x)$ tem $\partial f/\partial y = 2y$, que *não* é limitada em $\mathbb{R}$; logo f não é globalmente Lipschitz. Porém, ao longo da solução exata temos $-0{,}101 \le y \le 2$ em $[0, T]$, de modo que, numa vizinhança tubular da solução, f é Lipschitz com $L \approx \max |2y| = 4$. A demonstração usual se aplica enquanto a solução numérica permanecer nesse tubo, o que ocorre para k suficientemente pequeno. Para o Euler avançado,

$$|e_N| \;\le\; \frac{k}{2}\, \|y''\|_\infty \, \frac{e^{LT} - 1}{L} \;=\; 6\, \frac{e^{6{,}4} - 1}{4}\, k \;\approx\; 9 \times 10^2\, k, \tag{8}$$

(com $e_0 = 0$), e o Euler recuado satisfaz estimativa do mesmo tipo desde que $kL < 1$ (necessário para a unicidade da raiz do passo implícito). Consistência mais zero-estabilidade implicam, portanto, que **ambos os métodos convergem com ordem 1** quando $k \to 0$.

2.3 Implementação dos Métodos

Todos os métodos numéricos foram implementados em um mesmo módulo em Python (`src/pvi/metodos.py`). Ambas as rotinas recebem a instância que representa o PVI (contendo f, $\partial f/\partial y$, y_0 e t_0), o número de passos n e o comprimento do passo k. Como retorno, devolvem os vetores da malha $(t_i)_{i=0}^{m}$ e da solução numérica $(U_i)_{i=0}^{m}$ (onde $m \le n$ reflete eventuais interrupções por divergência), além de um dicionário com estatísticas computacionais: número total de avaliações de função (`nfev`) e tempo de CPU (`runtime`), mensurado via `time.perf_counter`.

Para mitigar a propagação de erros de arredondamento inerentes à aritmética de ponto flutuante, os nós da malha não são acumulados recursivamente ($t \leftarrow t + k$), mas sim computados diretamente por $t_i = t_0 + i \cdot k$.

2.3.1 Euler Explícito (Avançado)

No método de Euler explícito, cada passo requer uma única avaliação da função vetorial $f(t_n, U_n)$. A pré-alocação dos vetores em NumPy assegura eficiência de memória, e uma verificação condicional com `np.isfinite` interrompe o laço caso ocorra estouro numérico (*overflow*). A implementação em Python com comentários explicativos é apresentada em Código 1.

Listing 1 Implementação do método de Euler Avançado em Python.

```python
def euler_explicito(pvi, n, k):
    # Marca o início de execução para medição do tempo de CPU
    t0_cpu = time.perf_counter()
    # Pré-alocação dos vetores de tempo e solução numérica
    t_vals = np.zeros(n + 1)
    y_vals = np.zeros(n + 1)
    t_vals[0] = pvi.t0
    y_vals[0] = pvi.y0

    tn = pvi.t0
    Un = pvi.y0
    nfev = 0  # Contador de avaliações da função f
    # Contador de pontos válidos (para truncamento caso ocorra divergência)
    pts = 1

    for i in range(n):
        # Cálculo exato do instante tn+1 para evitar acúmulo de erro numérico
        tn1 = pvi.t0 + (i + 1) * k

        # Passo do Euler Explícito: U_{n+1} = U_n + k * f(t_n, U_n)
        Un1 = Un + k * pvi.f(tn, Un)
        nfev += 1
        # Interrupção preventiva caso a solução exploda (NaN ou Inf)
        if not np.isfinite(Un1):
            print(f"Euler explícito explodiu em t = {tn1:.4f} (com passo k = {k})")
            break

        t_vals[i+1] = tn1
        y_vals[i+1] = Un1
        tn = tn1
        Un = Un1
        pts += 1

    # Marca o fim da execução do método
    tf_cpu = time.perf_counter()
    # Empacotamento das estatísticas computacionais
    stats = {
        'nfev': nfev,
        'runtime': tf_cpu - t0_cpu,
    }
    return t_vals[:pts], y_vals[:pts], stats
```

2.3.2 Euler Implícito (Recuado)

Para o Euler implícito, cada passo exige determinar a raiz da equação não linear:

$$F(U) = U - U_n - k\, f(t_{n+1}, U) = 0. \tag{9}$$

A resolução é efetuada por meio do método de Newton (`scipy.optimize.newton`) com derivada analítica exata $F'(U) = 1 - k\,\frac{\partial f}{\partial y}(t_{n+1}, U)$, tolerância de convergência 10^{-12} e limite de 50 iterações.

Um ponto chave da implementação é o fornecimento da aproximação inicial: adota-se o preditor de Euler explícito, $U^{(0)} = U_n + k\, f(t_n, U_n)$. Por possuir erro local $O(k^2)$ em relação

à raiz procurada, esse chute garante convergência rápida e direciona o método estritamente para o ramo fisicamente consistente com o PVI. De fato, para $f(t, U) = U^2 - g(t)$, a equação (9) é quadrática:

$$k\,U^2 - U + c_n = 0, \qquad c_n := U_n - k\,g(t_{n+1}), \tag{10}$$

cujas raízes são

$$U_{n+1}^{\pm} = \frac{1 \pm \sqrt{\Delta_n}}{2k}, \qquad \Delta_n = 1 - 4k\,c_n. \tag{11}$$

Expandindo $\sqrt{1-4kc_n} = 1 - 2kc_n - 2k^2c_n^2 + O(k^3)$, verifica-se que a menor raiz satisfaz $U_{n+1}^{-} = U_n + k(U_n^2 - g(t_{n+1})) + O(k^2) \to U_n$ quando $k \to 0$ (ramo físico), enquanto $U_{n+1}^{+} = \frac{1}{k} - U_n + O(k) \to +\infty$ (ramo espúrio), resultando em $|U^{(0)} - U_{n+1}^{-}| = O(k^2)$. Além disso, como $F(U) = -kU^2 + U - c_n$ é côncava ($F''(U) = -2k < 0$) com máximo no vértice $U^* = \frac{1}{2k}$, qualquer chute inicial $U^{(0)} < \frac{1}{2k}$ situa-se à esquerda do vértice (onde $F'(U) > 0$), garantindo que as iteradas de Newton convergem monotonicamente para U_{n+1}^{-} sem saltar para U_{n+1}^{+}. O código fonte em Python é detalhado no Código 2.

Listing 2 Implementação do método de Euler Implícito com Newton em Python.

```python
def euler_implicito(pvi, n, k):
    t0_cpu = time.perf_counter()
    t_vals = np.zeros(n + 1)
    y_vals = np.zeros(n + 1)

    t_vals[0] = pvi.t0
    y_vals[0] = pvi.y0

    tn = pvi.t0
    Un = pvi.y0
    nfev = 0  # Contador de avaliações de funções (f e dfdy)

    pts = 1

    for i in range(n):
        # Instante t_{n+1} calculado diretamente da malha
        tn1 = pvi.t0 + (i + 1) * k

        # Resíduo da equação implícita: F(U) = U - U_n - k * f(t_{n+1}, U) = 0
        F  = lambda U: U - Un - k * pvi.f(tn1, U)
        # Derivada analítica dF/dU = 1 - k * df/dy(t_{n+1}, U) para o método de Newton
        dF = lambda U: 1 - k * pvi.dfdy(tn1, U)

        # Aproximação inicial via preditor de Euler Explícito: U^{(0)} = U_n + k * f(t_n, Un)
        U0 = Un + k * pvi.f(tn, Un)
        nfev += 1

        # Resolução da equação algébrica não linear via método de Newton
        Un1, info = newton(F, x0=U0, fprime=dF, tol=1e-12, maxiter=50,
                           full_output=True, disp=False)

        # Cada iteração do Newton avalia a função F e sua derivada dF
        nfev += info.iterations * 2

        # Interrupção se o método de Newton falhar em convergir ou gerar valor não finito
        if not info.converged or not np.isfinite(Un1):
            print(f"Euler implícito falhou em t = {tn1:.4f} (com passo k = {k})")
            break

        t_vals[i+1] = tn1
        y_vals[i+1] = Un1
        Un = Un1
        tn = tn1
        pts += 1

    # Marca o fim da execução do método
    tf_cpu = time.perf_counter()
    # Empacotamento das estatísticas computacionais
    stats = {
        'nfev': nfev,
        'runtime': tf_cpu - t0_cpu,
    }

    return t_vals[:pts], y_vals[:pts], stats
```

2.4 Resultados e Discussão

O experimento matemático rodado para este item foi o seguinte:

Integrou-se o PVI de Motivação (1) a partir de $t_0 = 0$ com condição inicial $y(0) = 2$ (script `experimentos/exp_a_fig122.py`), primeiramente no intervalo [0, 1,6] (conforme a Figura 12.2 de Süli e Mayers [1]) e, adicionalmente, no intervalo estendido [0, 5] para investigar a dinâmica em tempos mais longos. Empregaram-se os passos $k \in \{0{,}2;\ 0{,}1;\ 0{,}05\}$ (correspondentes a $n = 8, 16$ e 32 passos em $[0, 1{,}6]$), além de malhas mais refinadas ($k = 0{,}025;\ 0{,}0125;\ 0{,}00625$) para aferir o regime assintótico de convergência, registrando-se o erro máximo $E(k) = \max_i |U_i - y(t_i)|$, a ordem observada p_{obs}, o número de avaliações de função (NFEV) e o tempo de CPU.

2.4.1 Reprodução da Figura 12.2 no intervalo [0, 1,6]

A Figura 2 mostra as aproximações obtidas pelos dois métodos com $k = 0{,}2;\ 0{,}1$ e $0{,}05$. O painel da esquerda reproduz qualitativamente a Figura 12.2 de Süli e Mayers [1]. As Tabelas 1 e 2 complementam a figura com o erro máximo em $[0, 1{,}6]$ e a ordem observada $p_{\text{obs}} = \log_2\big(E(2k)/E(k)\big)$, incluindo passos menores para evidenciar o regime assintótico.

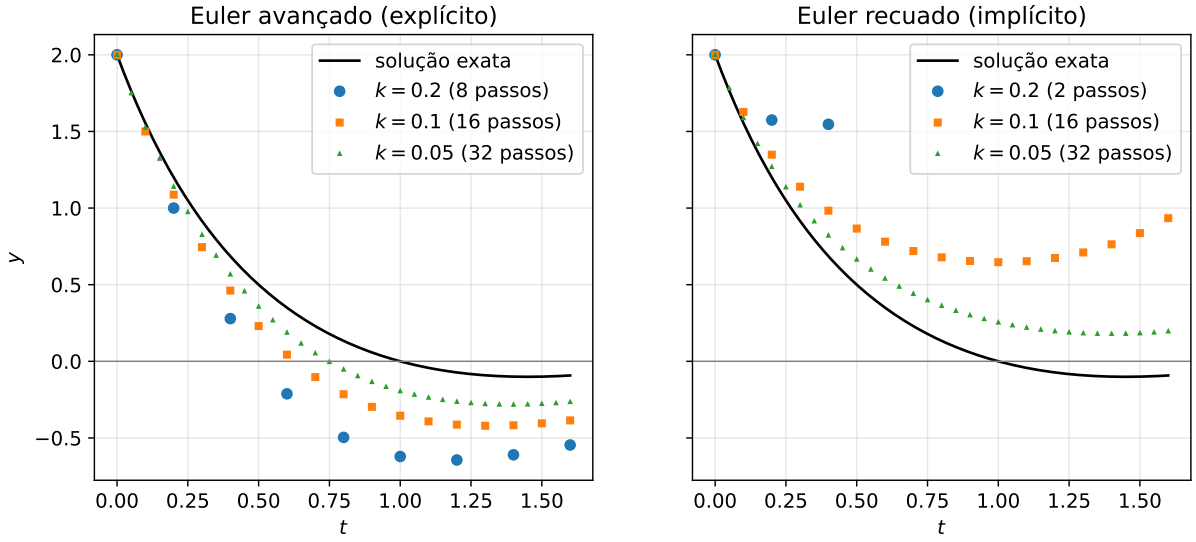


Figura 2: *Euler avançado (esquerda) e Euler recuado (direita) no intervalo* $[0, 1{,}6]$. *Com* $k = 0{,}2$ *o Euler recuado só consegue avançar 2 passos.*

k	Passos	NFEV	Tempo CPU (ms)	Erro Máximo	Ordem Observada
0.2	8	8	0.07	6.297e-01	-
0.1	16	16	0.04	3.549e-01	0.83
0.05	32	32	0.04	1.940e-01	0.87
0.025	64	64	0.08	1.027e-01	0.92
0.0125	128	128	0.24	5.310e-02	0.95
0.00625	256	256	0.55	2.704e-02	0.97

Tabela 1: *Análise computacional completa para o método Euler avançado.*

k	Passos	NFEV	Tempo CPU (ms)	Erro Máximo	Ordem Observada
0.2	2	123	1.40	8.607e-01	-
0.1	16	144	0.98	1.026e+00	-0.25
0.05	32	240	2.41	2.883e-01	1.83
0.025	64	446	2.87	1.218e-01	1.24
0.0125	128	868	5.21	5.763e-02	1.08
0.00625	256	1754	11.81	2.816e-02	1.03

Tabela 2: *Análise computacional completa para o método Euler recuado.*

A análise conjunta da Figura 2 e das Tabelas 1 e 2 evidencia quatro fenômenos teóricos fundamentais:

1. Posição geométrica das trajetórias e sinal de $y''(t)$. Derivando duas vezes a solução exata $y(t) = t - 4 + \frac{6}{1+t}$, obtém-se

$$y''(t) = \frac{12}{(1+t)^3} > 0, \qquad \forall\, t > -1. \tag{12}$$

Como a solução exata é estritamente convexa em todo o intervalo, as expressões do erro de truncamento local (7) explicam imediatamente a disposição visual dos pontos na Figura 2: no Euler avançado, a aproximação pela reta tangente em t_n subestima a curva convexa ($\tau_n^{\mathrm{FE}} > 0 \implies U_n < y(t_n)$), posicionando todas as trajetórias numéricas *abaixo* da solução exata. No Euler recuado, o uso da inclinação futura em t_{n+1} superestima a solução ($\tau_n^{\mathrm{BE}} < 0 \implies U_n > y(t_n)$), posicionando todas as trajetórias *acima* da curva exata.

2. Explicação algébrica da falha do Euler recuado em $k = 0{,}2$. As Tabelas 3 e 4 mostram a evolução passo a passo dos dois métodos para $k = 0{,}2$.

i	t_i	Numérico (U_i)	Exato ($y(t_i)$)	Erro ($\lvert U_i - y(t_i)\rvert$)
0	0.0000	2.000000	2.000000	0.000e+00
1	0.2000	1.000000	1.200000	2.000e-01
2	0.4000	0.278667	0.685714	4.070e-01
3	0.6000	-0.212088	0.350000	5.621e-01
4	0.8000	-0.496342	0.133333	6.297e-01
5	1.0000	-0.620997	0.000000	6.210e-01
6	1.2000	-0.643869	-0.072727	5.711e-01
7	1.4000	-0.609947	-0.100000	5.099e-01
8	1.6000	-0.545874	-0.092308	4.536e-01

Tabela 3: *Evolução passo a passo para o método Euler avancado com passo $k = 0.2$.*

i	t_i	Numérico (U_i)	Exato ($y(t_i)$)	Erro ($\lvert U_i - y(t_i)\rvert$)
0	0.0000	2.000000	2.000000	0.000e+00
1	0.2000	1.574437	1.200000	3.744e-01
2	0.4000	1.546458	0.685714	8.607e-01

Tabela 4: *Evolução passo a passo para o método Euler recuado com passo $k = 0.2$.*

Para que a equação do Euler recuado (10) admita solução real no passo i, é necessário e suficiente que

$$\Delta_{i-1} = 1 - 4k\,c_{i-1} \geq 0 \iff c_{i-1} = U_{i-1} - k\,g(t_i) \leq \frac{1}{4k}.$$

Para $k = 0{,}2$, esse limiar crítico é $\frac{1}{4k} = 1{,}25$. Nos dois primeiros passos ($t_1 = 0{,}2$ e $t_2 = 0{,}4$), o erro de truncamento positivo combinado à amplificação não linear mantém $U_2 \approx 1{,}5465$ muito acima do valor exato $y(0{,}4) \approx 0{,}6857$. Ao avançar para $t_3 = 0{,}6$ (onde $g(0{,}6) = 1{,}46625$), obtém-se

$$c_2 = U_2 - 0{,}2\,g(0{,}6) \approx 1{,}546458 - 0{,}293250 = 1{,}253208 > 1{,}25 \implies \Delta_2 \approx -0{,}002566 < 0.$$

Completando quadrados no resíduo (10), vê-se que

$$F(U) = -0{,}2\,(U - 2{,}5)^2 - 0{,}003208 \;\leq\; -0{,}003208 < 0, \qquad \forall\, U \in \mathbb{R}.$$

Portanto, a parábola $F(U)$ situa-se inteiramente abaixo do eixo real e possui apenas raízes complexas conjugadas ($U_3 = 2{,}5 \pm 0{,}1266\,i$). Fica provado que a falha em $k = 0{,}2$ *não* decorre de má escolha do chute inicial ou do algoritmo de Newton, mas sim da *inexistência de solução real* do esquema implícito quando o passo k viola a barreira de solubilidade $k\,\frac{\partial f}{\partial y} < 1$ (polo $z = 1$ de $R_{\mathrm{BE}}(z)$). Uma busca numérica em malhas uniformes $k = 1{,}6/n$ mostra que o maior passo que permite ao Euler recuado completar $[0, 1{,}6]$ é $k = 1{,}6/13 \approx 0{,}1231$ ($n = 13$ passos).

4. Ordem de convergência e custo computacional. Nas Tabelas 1 e 2, à medida que $k \to 0$, a ordem empírica p_{obs} de ambos os métodos converge para o valor teórico $p = 1$ (0,97 no explícito e 1,03 no implícito em $k = 0{,}00625$). O valor aparente $p_{\mathrm{obs}} = 1{,}83$ na transição de $k = 0{,}1$ para $k = 0{,}05$ no Euler recuado não significa precisão de segunda ordem: ele ocorre porque $k = 0{,}1$ está próximo do limiar de perda de existência de raiz ($k \approx 0{,}123$), sofrendo forte amplificação pré-assintótica. Quanto ao custo, o Euler recuado exigiu em média de 3 a 4 iterações de Newton por passo (totalizando 7 a 9 avaliações de função por passo contra apenas 1 do Euler avançado) e um tempo de CPU substancialmente superior, sendo simultaneamente mais custoso e menos preciso neste problema.

2.4.2 Comportamento em Longo Prazo: Extensão para o Intervalo $[0, 5]$

Para investigar como a estabilidade não linear afeta a integração em intervalos mais longos — e ainda mais distantes da singularidade em $t = -1$ —, a Figura 3 e as Tabelas 5 e 6 apresentam os resultados para $t \in [0, 5]$.

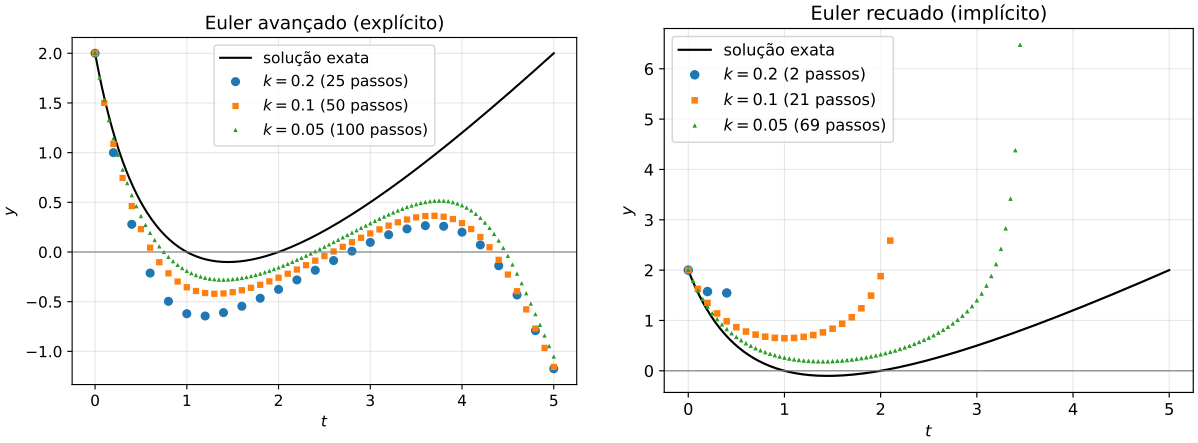


Figura 3: *Soluções numéricas no intervalo estendido* $[0, 5]$ *para o Euler avançado (esquerda) e Euler recuado (direita).*

k	Passos	NFEV	Tempo CPU (ms)	Erro Máximo	Ordem Observada
0.2	25	25	0.04	3.175e+00	-
0.1	50	50	0.06	3.157e+00	0.01
0.05	100	100	0.12	3.056e+00	0.05
0.025	200	200	0.25	2.857e+00	0.10
0.0125	400	400	0.49	2.526e+00	0.18
0.00625	800	800	1.03	2.054e+00	0.30

Tabela 5: *Análise computacional completa para o método Euler avançado,* $TF = 5$.

k	Passos	NFEV	Tempo CPU (ms)	Erro Máximo	Ordem Observada
0.2	2	123	1.09	8.607e-01	-
0.1	21	290	2.07	2.551e+00	-1.57
0.05	69	620	4.07	5.673e+00	-1.15
0.025	164	1267	8.50	1.692e+01	-1.58
0.0125	359	2582	20.93	1.907e+01	-0.17
0.00625	766	5223	38.15	4.151e+01	-1.12

Tabela 6: *Análise computacional completa para o método Euler recuado,* $TF = 5$.

Os resultados em $[0, 5]$ expõem o papel da solução exata $y(t)$ como uma **separatriz instável** (curva repulsora) da equação de Riccati $y' = y^2 - g(t)$:

- **Explosão em tempo finito (*blow-up*) no Euler Recuado:** Como o erro local mantém $U_n > y(t_n)$, quando a solução exata volta a ficar positiva ($t > 2$), a perturbação positiva $e = U - y > 0$ entra no regime dominado pelo termo quadrático $e' = 2y\,e + e^2 \approx e^2$, que possui assíntota vertical em tempo finito. Em todas as malhas testadas, U_n dispara até violar a condição $\Delta_n \geq 0$ (sendo necessário reduzir o passo até $k \leq 5/1462 \approx 0{,}00342$ para que o Euler recuado alcance $t = 5$).

- **Captura por atrator espúrio no Euler Avançado:** Embora o método explícito não exploda, suas trajetórias sofrem uma bifurcação qualitativa severa. No trecho

$t \in (1{,}462;\ 3{,}508)$, tem-se $g(t) < 0$, de modo que $f(t, U) = U^2 - g(t) > 0$ para todo U, fazendo as soluções numéricas subirem acompanhando (por baixo) a solução exata. Contudo, para $t > 3{,}508$, $g(t)$ volta a ser positiva e surgem as curvas isóclinas de derivada nula $U = \pm\sqrt{g(t)}$. Enquanto a solução exata permanece acima da isóclina repulsora $+\sqrt{g(t)}$ e continua crescendo até $y(5) = 2$, as aproximações do Euler explícito — por estarem defasadas para baixo — são interceptadas pela curva $U = +\sqrt{g(t)}$ em torno de $t \approx 3{,}6$–$3{,}8$. A partir desse cruzamento, $U_n^2 < g(t_n) \implies f(t_n, U_n) < 0$: a derivada muda de sinal, as trajetórias atingem um máximo local e mergulham para a região $U < 0$, onde $\frac{\partial f}{\partial y} = 2U < 0$ é fortemente atrativa, sendo capturadas pelo ramo assintoticamente estável (porém incorreto) $U(t) \approx -\sqrt{g(t)}$.

2.4.3 Síntese Crítica: Comparação entre Euler Explícito e Implícito

A investigação realizada neste item permite formular três conclusões críticas que desmistificam a aplicação automática de métodos implícitos em PVIs não lineares:

1. **Irrelevância da distância à singularidade frente à instabilidade intrínseca:** Mesmo integrando em $[0, 1{,}6]$ ou $[0, 5]$ — regiões onde a solução exata $y(t)$ é perfeitamente suave e afastada do polo em $t = -1$ —, ambos os métodos enfrentam severa degradação. A proximidade da singularidade em $t = 0$ apenas eleva a curvatura inicial ($y''(0) = 12$), mas o fator determinante para a divergência subsequente é a instabilidade dinâmica da própria EDO ($\frac{\partial f}{\partial y} = 2y > 0$).

2. **A A-estabilidade não protege contra modos crescentes nem contra erros no lado repulsivo:** O Euler implícito é incondicionalmente estável apenas quando $\frac{\partial f}{\partial y} < 0$. Neste PVI, a combinação de convexidade ($y'' > 0$, que força o Euler implícito a errar para cima) com Jacobiana positiva ($\frac{\partial f}{\partial y} = 2y > 0$) empurra a aproximação implícita diretamente para a bacia de explosão (*blow-up*) da equação de Riccati e amplifica o erro a uma taxa $(1-z)^{-1} > e^z$. Já o Euler explícito, ao errar para baixo, beneficia-se do amortecimento na região $U + y < 0$ em $[0, 1{,}6]$, mas em intervalos longos ($[0, 5]$) acaba capturado pelo atrator espúrio $-\sqrt{g(t)}$.

3. **Restrição de passo por solubilidade no método implícito:** Enquanto métodos explícitos sofrem restrição de passo por estabilidade em problemas rígidos ($\lambda \ll 0$), o Euler implícito sofre aqui uma restrição severa de passo ($k < 0{,}123$ em $[0, 1{,}6]$ e $k < 0{,}00342$ em $[0, 5]$) para garantir a própria *existência de raiz real* ($\Delta_n \geq 0$, ligada a $k\,\frac{\partial f}{\partial y} < 1$). Assim, com ordem de precisão baixa ($p = 1$), o pagamento do alto custo computacional das iterações de Newton no Euler recuado não traz qualquer benefício, motivando diretamente o emprego de métodos de ordem superior nos itens (b) e (c).

3 Item (b): Método Explícito de Runge-Kutta de Quarta Ordem

3.1 Objetivo

Implementar o método explícito clássico de Runge-Kutta de quarta ordem (RK4), conforme descrito no Exemplo 5.13 de LeVeque [2] (p. 126), resolver novamente o PVI de Motivação (1) e apresentar uma descrição crítica comparativa entre o método explícito RK4 e o método explícito de Euler avançado, discutindo suas vantagens e desvantagens computacionais e teóricas.

3.2 Fundamentação Teórica

Consistência

Estabilidade

3.3 Implementação do Método

O método RK4 foi implementado na função `rk4` dentro do mesmo módulo `src/pvi/metodos.py`, seguindo exatamente a mesma interface das rotinas de Euler. Em cada iteração do laço principal, calculam-se sequencialmente os quatro estágios explícitos (??)–(??), contabilizando exatamente 4 avaliações da função f por passo (`nfev += 4`), sem necessidade de derivada $\partial f/\partial y$ nem de solucionadores iterativos. O código em Python comentado é exibido no Código 3.

Listing 3 Implementação do método explícito de Runge-Kutta de 4ª Ordem (RK4) em Python.

```python
def rk4(pvi, n, k):
    # Marca o início de execução para medição do tempo de CPU
    t0_cpu = time.perf_counter()

    # Pré-alocação dos vetores de tempo e solução numérica
    t_vals = np.zeros(n + 1)
    y_vals = np.zeros(n + 1)
    t_vals[0] = pvi.t0
    y_vals[0] = pvi.y0

    tn = pvi.t0
    Un = pvi.y0
    nfev = 0  # Contador de avaliações da função f

    # Contador de pontos válidos (para truncamento caso ocorra divergência)
    pts = 1

    for i in range(n):
        # Estágio 1: inclinação no início do passo (tn)
        Y1 = Un
        F1 = pvi.f(tn, Y1)

        # Estágio 2: estimativa no ponto médio (tn + k/2) usando F1
        Y2 = Un + 0.5 * k * F1
        F2 = pvi.f(tn + 0.5 * k, Y2)

        # Estágio 3: nova estimativa no ponto médio (tn + k/2) usando F2
        Y3 = Un + 0.5 * k * F2
        F3 = pvi.f(tn + 0.5 * k, Y3)

        # Estágio 4: estimativa no final do passo (tn + k) usando F3
        Y4 = Un + k * F3
        F4 = pvi.f(tn + k, Y4)

        # Combinação ponderada (regra de Simpson) para obter U_{n+1}
        Un1 = Un + (k / 6.0) * (F1 + 2*F2 + 2*F3 + F4)
        tn1 = pvi.t0 + (i + 1) * k

        # O método RK4 clássico avalia a função f exatamente 4 vezes por passo
        nfev += 4

        # Interrupção preventiva caso a solução exploda (NaN ou Inf)
        if not np.isfinite(Un1):
            print(f"RK4 explodiu em t = {tn1:.4f} (com passo k = {k})")
            break

        t_vals[i+1] = tn1
        y_vals[i+1] = Un1
        tn = tn1
        Un = Un1

        pts += 1

    # Marca o fim da execução do método
    tf_cpu = time.perf_counter()

    # Empacotamento das estatísticas computacionais
    stats = {
        'nfev': nfev,
```

3.4 Resultados e Discussão

4 Item (c): Método Implícito de Adams-Moulton de Dois Passos

4.1 Objetivo

Implementar o método implícito de múltiplos passos lineares de Adams-Moulton de 2 passos (*2-step Adams-Moulton*), conforme apresentado na página 132 de R. J. LeVeque [2], resolver novamente o PVI de Motivação (1) e apresentar uma descrição crítica comparativa entre os dois métodos implícitos analisados — o método de Adams-Moulton de 2 passos e o método de Euler recuado —, discutindo suas vantagens e desvantagens teóricas e computacionais.

4.2 Fundamentação Teórica

Consistência

Estabilidade

4.3 Implementação do Método

4.4 Resultados e Discussão

5 Item (d): Análise Crítica Geral — Métodos Explícitos versus Implícitos na Resolução de PVIs

5.1 Objetivo

Integrar criticamente os resultados teóricos e computacionais obtidos nos Itens (a), (b) e (c) para discutir as vantagens, desvantagens e critérios de seleção entre **Métodos Explícitos** e **Métodos Implícitos** na resolução numérica de Problemas de Valor Inicial (PVIs) de um modo geral. Adicionalmente, investigar teórica e experimentalmente o que ocorre ao tentar resolver o PVI de Motivação (1) nos intervalos $[0, 5]$, $[-2, 2]$ e $[-5, 40]$ (bem como $[0, 40]$).

Referências

[1] E. Süli e D. F. Mayers. *An Introduction to Numerical Analysis*. Cambridge University Press, 2003.

[2] R. J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM, 2007.